HIBERNATE

Hibernate is an ORM (Object-Relational Mapping) framework for Java that maps Java objects to relational database tables and handles database operations automatically.

Advantages:

- Reduces boilerplate JDBC code.
- Database-independent (uses HQL).

Hibernate configuration file: (hibernate.config.xml or hibernate.properties)

- This file contains database-related configuration and session-related configuration.
- JDBC - Java Database Connectivity
  Advantages of Hibernate over JDBC

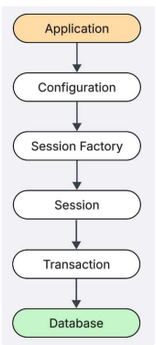
JDBC	HIBERNATE
Manual SQL	Automatic SQL generation
More code	Less code
No Cache	Has Cache
DB dependent	DB independent

- Lazy loading: Data is loaded only when required, not upfront.
  It is mainly used to improve application performance by loading child objects only when the parent object is accessed.
- Eager loading: Data is loaded upfront by loading child objects along with the parent object.

Hibernate Architecture

- Workflow of Hibernate

  1. Application loads configuration
  2. `Configuration` object created
  3. `SessionFactory` is built
  4. `Session` is opened
  5. Transaction begins
  6. Perform operations (`save`, `update`)
  7. Hibernate generates SQL
  8. Executes SQL on DB
  9. Transaction commit
  10. Session closed



- Architecture of Hibernate:
  1. Application Layer: Our Java application(DTO, Entity & Service classes) calls hibernate API's to perform operations.
  2. Configuration Object: Reads hibernate configuration(hibernate.cfg.xml) file contains db details like url, username, dialect.
  3. Session Factory
  4. Session
  5. Transaction
  6. Query
  7. Database

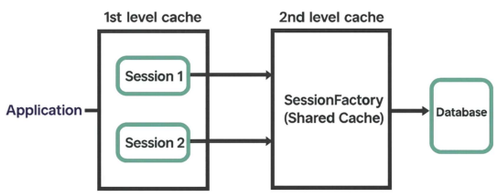
Important interfaces in Hibernate

- SessionFactory: Heavy weighted object created once per application; used to create Session objects.
- Session: Main interface for db interaction. Light weight created once per request. Maintains the connection between Java objects and the database.
- Transaction: Represents a unit of work; handles operations with commit() and rollback().
- Query: Used to create and execute HQL or SQL queries.
- Database: Final Storage. Hibernate converts Objects → SQL → database.
- SessionFactory Builder: Builds a SessionFactory based on configuration settings.

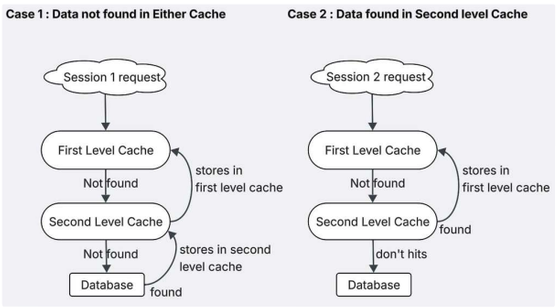
- **Diff b/w get() & load() :**
  - get() hits db immediately & returns **actual object or null if not found.**
  - load() lazy loading, doesn't hit db immediately & returns **proxy object or throws exception if not found.**
- Cache: A temporary memory area where frequently accessed data is stored so repeated queries can be served faster without hitting the database. This improves Hibernate performance by reducing the number of times it needs to communicate with the DB.
- **Hibernate uses 2 levels of caching :**

First Level Cache(Session level)	Second Level Cache(Session Factory level)
This Cache is local to the session object & can't be shared with different sessions.	This Cache is maintained at SessionFactory level & main benefit from this cache is different sessions can share cached entities to avoid db queries.
Available only until session is open once closed, cache will be destroyed.	Available throughout the application. It is only destroyed & recreated when application is restarted.

- **Entity lookup order :**



- **<u>If Session requests an entity, hibernate follows this order :</u>**



- **Jshell/Spring shell :** If we want to interact with Hibernate API's & database entities directly in terminal without compiling full application we can use Jshell / Springshell.
- **JPA(Java Persistence API) :** JPA is a java specification acts as bridge b/w java objects & database records, makes easier to work with relational databases.

  It maps Java objects to database tables & manage data using simple API's instead of writing complex SQL.
- **Features of JPA in Java :**
  1. Allows direct mapping : Maps java classes & fields directly to database tables & columns.
  2. Portable : Works across different databases without changing the code.
- **Entity Manager :** acts as the primary API for database interactions & ensures entities are persisted properly.
- **merge() :** Used to update a detached entity back into persistence context.
- **Entity states in Hibernate :**
  1. **Transient :** Newly created, not persisted.
  2. **Persistent :** Managed by a hibernate session.
  3. **Detached :** Was persistent, but session is closed.
  4. **Removed :** Deleted from database & no longed managed.
- **Dialect :** Specifies the database type(eg. Oracle, Mysql, etc.). So, hibernate can generate correct SQL.
- **Cascading :** Used to propogate(do) operations(eg. save(), delete()) from parent to child.
- **flush() & why ?**

  → used to sync session data with database immediately.

  → because, hibernate does not execute SQL immediately when we call (save(), delete(),etc) methods. Instead it stores in session(first level cache) & executes them later. **flush()** forces hibernate to send those pending SQL statements to database immediately.

- **What happens if session is not closed ?**

  → Memory leak, Connections will exhaust.
- **What exactly is N+1 problem ?**

  N+1 problem occurs when hibernate runs 1 query to fetch parent & N queries to fetch child leads to performance issue.
- **How do you improve hibernate performance ?**

  → Use cache

  → Use LazyLoading.

  → Avoiding N+1 problem.

  → Optimizing queries.

- **Types of Relationships b/w entities :**
  1. **One to One(1:1) :** One object is related to exactly one object.

     eg.: **Person → Passport**
  2. **One to Many(1:N):** One object is related to many objects.

     eg.: **Department → Employees**
  3. **Many to One(N:1):** Many objects are related to one object.

     eg.: **Employees → Department**
  4. **Many to Many(N:N):** Many objects are related to many objects.

     eg.: **Students ↔ Courses**